

Top 50 Angular Interview Questions and Answers

Study notes from the YouTube course:

<https://www.youtube.com/watch?v=4KBVkJ7b6yk>

Length about 2 hours 13 minutes. Instructor introduces himself as Happy (Interview Happy style course). These notes are cleaned from the video captions so you can read instead of watching.

Topics in this course: Angular framework, components and modules, data binding, directives, decorators and pipes, services and dependency injection, lifecycle hooks, Observable / HTTP Client / RxJS, TypeScript basics.

1. Angular framework

Q1. What is Angular?

Angular is a **component-based framework** for building structured, scalable, **single-page** client-side web applications.

The browser understands HTML, CSS, and JavaScript. Servers use languages like Java, C#, PHP. Large apps need a framework on the client too, the same way Spring or .NET structure the server. Angular is that client-side framework.

SPA: one page; only inner content updates. YouTube is an SPA (the reload icon does not become a full page reload). A multi-page site reloads the whole page on each menu click.

Scalable means open to later enhancement without rewriting everything.

Q2. Advantages of Angular / why Angular?

- SPAs are simpler than in plain JavaScript because of component architecture.
- Structure for large client apps (OOP: classes, interfaces).
- Cross-platform and open source.
- Reusable code via **services**.
- Testability: `.spec.ts` files for unit tests.

You can build an SPA in plain JS, but Angular makes structure, reuse, and tests much easier.

Q3. Angular vs AngularJS?

Google released AngularJS in 2010, then rewrote the framework as **Angular** (dropped “JS” because it is not JavaScript-only; it uses TypeScript).

- AngularJS: JavaScript only. Angular: TypeScript + JavaScript; compile-time errors; classes/interfaces.
- AngularJS: MVC. Angular: **component-based** (faster).
- Angular has **CLI**; AngularJS does not.
- Angular uses **dependency injection**.
- Angular supports mobile browsers better.
- Angular is faster (data binding + components).

Q4. What is npm? Why install Node for Angular?

npm = Node Package Manager: an online library of packages. Google built Node.js and Angular, so they share npm. Install Node first so you get npm. Downloaded packages live in `node_modules`.

Q5. What is the Angular CLI?

Command-line tool to create projects, components, services, modules. Example: `ng generate component test` scaffolds the component files under `src/app`.

Definition: CLI initializes and develops Angular applications (new project, generate files, serve, build).

2. Components and modules

Q6. What are components?

The basic UI building block. One page can have many components (menu, login, product list). The page stays; components swap or refresh.

A component is four files together: `.css`, `.html`, `.spec.ts`, `.ts`. The `.ts` file holds `@Component` metadata: selector, templateUrl, styleUrls. A new project starts with `AppComponent`.

Q7. Selector and template?

Selector: unique name in the component tree (e.g. `app-root`). In `index.html`, that tag is replaced by the component's HTML.

Template: the HTML view. `templateUrl` points to a file, or `template` for inline HTML.

Q8. What is a module? What is `app.module.ts`?

A module groups related components, directives, pipes, and services. Root module is `AppModule` with `@NgModule`:

- **declarations** — components in this module
- **imports** — other modules (e.g. `BrowserModule` to run in the browser)
- **exports** — make this module usable by others
- **providers** — DI (covered later)
- **bootstrap** — which declared component starts first

Small apps: one root module. Large apps: more modules.

Q9. How does an Angular app start? `index.html`, `app-root`, `main.ts`

Sequence to remember:

1. Browser loads **`index.html`** (the single page).
2. CLI loads JS bundles including **`main.js`** (from `main.ts`).
3. `main.ts` is the entry point: compiles and `bootstrapModule(AppModule)`.
4. `AppModule` bootstraps **`AppComponent`** (`app-root`).
5. `app-root` in `index.html` is replaced by `app.component.html`.

The “Loading...” text in `index.html` disappears because `app-root` is not a native HTML tag; Angular replaces it.

Q10. Bootstrap module and bootstrap component?

First module launched = bootstrap module (`bootstrapModule` in `main.ts`, usually `AppModule`).
First component(s) launched = bootstrap array in that module. You can bootstrap more than one component (e.g. `AppComponent` + `HeaderComponent`).

3. Data binding

Q11. What is data binding?

Communication between TypeScript (component class) and the HTML view.

- Component → view: interpolation, property binding (one-way out)
- View → component: event binding (one-way in)
- Both ways: two-way binding

People often say “component” for the `.ts` file even though a component is `ts + html + css`.

Q12. String interpolation?

One-way: component → view. Syntax: `{{ title }}`. Works with **strings**. Simple syntax, used for most display text.

Q13. Property binding?

Also component → view. Syntax: `[disabled]="isDisabled"`. Can bind **any type** (boolean, etc.). Interpolation cannot set `disabled`. Use interpolation for strings (simpler); property binding when the target is not a string.

Q14. Event binding?

View → component. Syntax: `(click)="onClick()"` (round brackets). Handles user events (click, mouse, etc.). You can pass arguments from the template into the method.

Q15. Two-way data binding? (very important)

View ↔ component at the same time. Syntax: [(ngModel)]="name" — “banana in a box”: [] property + () event. Needs `FormsModule` in imports. Type in an input and a {{ name }} updates immediately. Knowing this usually means you know all four bindings.

4. Directives

Q16. What are directives? Types?

Classes that add behavior to elements.

- **Structural** — add/remove DOM: `*ngIf`, `*ngFor`, `*ngSwitch`
- **Attribute** — change look/behavior: `ngClass`, `ngStyle`
- **Component** — directive with its own template (every component is this)

You can also write custom directives.

Q17. *ngIf?

Add or remove elements based on a condition (e.g. show “2 students” if length > 0, else “0 students”).

Q18. *ngFor?

Iterate a list and render each item (like a for loop) with interpolation inside.

Q19. *ngSwitch?

Used with `*ngSwitchCase` and `*ngSwitchDefault`. Like switch/case: matching case shows; default if none match.

Q20. ngStyle?

Attribute directive that sets styles. Benefit over static HTML style: styles can come from the component at runtime, e.g. `[ngStyle]='{'background-color': colorName}'`.

Q21. ngClass?

Adds/removes CSS classes. Cleaner than listing many styles: `[ngClass]='className'` where the class is defined in the component CSS.

Q22. Component vs structural vs attribute directives?

- Component: first view (`@Component`, has a template).
- Structural: add/remove elements (* prefix: `*ngIf`).
- Attribute: appearance (`[ngClass]`, `[ngStyle]`).

They work together to build an editable view.

5. Decorators and pipes

Q23. What is a decorator?

Stores **metadata** about a class, method, or property. Metadata is data about data (book title/author vs the book text). `@Component` metadata: selector, templateUrl, styleUrls. `@NgModule`: declarations, imports, providers, bootstrap. Decorators use `@`.

Q24. Types of decorators?

Class, property, method, parameter. Important class decorators: `@NgModule`, `@Component`. All start with `@`.

Q25. What are pipes?

Functions that take a value and return a transformed value. Built-in (lowercase, uppercase, date, currency, ...) or custom. Syntax: `{{ title | lowercase }}`.

Q26. Parameterized pipes?

Pass arguments after a colon: `{{ amount | currency: 'INR' }}` vs default USD.

Q27. Chaining pipes?

Multiple pipes in a row: date then uppercase, e.g. `{{ dob | date | uppercase }}`.

6. Services and dependency injection

Q28. Explain services (example).

A service is a TypeScript class of **reusable** code injected into many components. Example: error-logging in login and menu — put it once in `LoggingService` instead of copying it 100 times. Also used to centralize data (user data service). Implemented with **DI**.

Q29. How to create a service?

ng generate service logging (often in a services folder). Do **not** new `LoggingService()` in the component — that hurts unit tests. Always inject.

Q30. How to use DI with a service? (3 steps)

1. Register a **provider** (tell Angular the service exists).

2. Constructor parameter: `constructor(private logging: LoggingService).`
3. Call `this.logging.logError()`.

Q31. Hierarchical dependency injection?

If a parent provides a service, children can use it. If a child provides it, parent/siblings do not get that instance.

Three provider places:

- **Component** — this component + children
- **Module** — all components in the module
- **providedIn: 'root'** on `@Injectable` (Angular 8+ default) — whole app

Q32. What is a provider?

An object in a decorator that tells Angular a service can be injected. Same three levels as above.

Q33. Role of `@Injectable`? Service inside another service?

`@Injectable` lets a service be injected into components **and** other services. Both services need it. Example: `LoggingService` depends on `WritingService` via constructor. Remove `@Injectable` from `WritingService` → `NullInjectorError`. Remove it from `LoggingService` → compile error. Put it on both.

7. Lifecycle hooks

Q34. Parent vs child components?

`App`/`root` is usually the parent. Put a child's selector in the parent template (same idea as `app-root` in `index.html`). Generate child with `ng g c child`, copy selector into parent HTML.

Q35. What are lifecycle hooks?

Stages from create to destroy: instantiate, render, create children, destroy. You put code in hooks to run at those times.

Main ones: constructor (not officially a hook, but always mentioned), ngOnChanges, ngOnInit, ngDoCheck, child hooks (ngAfterContentInit, etc.), ngOnDestroy.

Q36. Constructor?

TypeScript class method, runs on instantiate, **before any Angular hook**, not part of the hook list. Main use: **inject services**. Keep it thin.

Q37. ngOnInit?

Most important hook. Means the component is created; put business logic here (load data, subscriptions). Called once, after ngOnChanges. Generated by default in new components.

Q38. Constructor vs ngOnInit? (favorite difference question)

- ngOnInit = Angular hook after the component is ready. Constructor = TypeScript, class is only instantiating.
- Constructor runs before any hook (even ngOnChanges). ngOnInit runs after ngOnChanges.
- **When to use:** constructor = inject dependencies (they do not need the view). ngOnInit = business logic when the component is “mature”. Logic in constructor is like putting job stress on a newborn — things are not ready, the app can break.

8. Observable, HTTP client, RxJS

Q39. Synchronous vs asynchronous?

JavaScript is single-threaded: tasks wait in line (slow). Async lets work overlap. JS uses Promises; TypeScript/Angular also uses **Observables**.

Q40. Promise vs Observable? (very important)

Both move data asynchronously.

- Promise: one value when **all** data is ready (client waits for 1 million rows).
- Observable: **streams** many values over time (chunks).
- Observables are **lazy**: nothing runs until `subscribe()`. Promises run as soon as created.
- Subscriptions can be **unsubscribed**. Promises cannot be cancelled.

Newspaper analogy: do not print papers if nobody subscribes.

Q41. What is RxJS?

Reactive Extensions for JavaScript. Library for async and event-based programs using observable sequences. Angular did **not** invent Observables; RxJS is the “father.” Any JS app can use them. Two players: **Observable** (emits) and **Observer/Subscriber** (receives).

Q42. How to implement an Observable? (3 steps)

1. Import Observable from `rxjs`.
2. Create it and `observer.next('A')` etc.
3. `subscribe()` in the component.

No subscribe → no emit (lazy). A Promise still starts without a subscriber.

Q43. Role of HttpClient?

Built-in Angular service for HTTP. Use `get` (and `post`, etc.) with Observables to talk to APIs. Real apps: `HttpClient` in a service → `Observable` → components `subscribe`.

9. TypeScript basics

Q44. What is TypeScript? Advantages over JavaScript?

Superset of JS (valid JS is valid TS). Browser does not run TS; Angular compiles TS → JS.

- Strongly typed vs JS loosely typed
- OOP: classes, interfaces
- Errors at **compile time**, not only runtime

Example: a `string` variable cannot be assigned a number in TS; JS allows it and may break later.

Q45. `let` vs `var`?

`var`: function-scoped, leaks out of a `for` block. `let`: block-scoped. Prefer `let`.

Q46. Type annotation?

Declaring the type: `let x: number = 1`. Wrong types fail at compile time.

Q47. Primitive vs non-primitive types?

Primitive/built-in: `boolean`, `number`, `string`. User-defined/complex: `array`, `tuple`, `enum`, `class`. Arrays are one type; tuples mix types in fixed positions.

Q48. `any`?

Turns off type checking. Uninitialized variables are implicitly `any`; better to write `any` explicitly if you must. Tip: English meaning of “any” = anything — that is the type.

Q49. `enum`?

Named constants (e.g. `Direction.East`). Auto numbers from 0, or set a start value. Readable and usable across files.

Q50. Type assertion and arrow functions?

Assertion: tell the compiler the type when it cannot infer it, e.g. `(<string>myName).length`, so you get string methods.

Arrow functions: short anonymous functions (like C# lambdas): `(a, b) => a * b`.

Closing from the video

The instructor's advice: attend as many interviews as you can, and do not give up. Use the description timestamps to jump to a question. Several items are marked "very important": data binding (especially two-way), directives differences, services/DI, `ngOnInit` vs constructor, Promise vs Observable.

Source: auto captions of the YouTube video, rewritten as study notes. Wording is cleaned; it is not a word-for-word legal transcript.